Table des matières

Mécanismes de sécurité de la base de données - moyun-pos 1
1. Mécanismes au niveau SQLite . . . 1
2. Mécanismes au niveau applicatif . . . 2
3. Mécanismes au niveau système . . . 2
4. Mécanismes au niveau cryptographique . . . 4
5. Sauvegardes . . . 4
6. Confidentialité (GDPR) . . . 4
7. Audit trail croisé . . . 4

Mécanismes de sécurité de la base de données - moyun-pos

Conformément au §II.C du Manuel de certification SCE 2.0, ce document décrit les mécanismes techniques et organisationnels qui protègent la base de données fiscale contre la modification, la suppression et l'accès non autorisé.

1. Mécanismes au niveau SQLite

1.1 WAL (Write-Ahead Logging)

```
PRAGMA journal_mode = WAL;
```

Activé au démarrage par `pos_db::migrate::run_migrations` (`crates/pos-db/src/migrate.rs:62`).

Le mode WAL garantit : - Atomicité crash-safe : un événement fiscal n'est commité que lorsque toute la transaction (compteur + archive + clôture commande) a été inscrite dans le WAL et synchronisée sur disque. - Cohérence en cas de coupure de courant : SQLite rejoue ou abandonne automatiquement les transactions partielles au prochain démarrage. Aucun événement n'est jamais à moitié écrit.

1.2 Foreign keys

```
PRAGMA foreign_keys = ON;
```

Activé au démarrage. Empêche toute insertion `order_lines.order_id` ou `fiscal_events.order_id` qui pointerait sur une commande inexistante.

1.3 Contraintes CHECK

Au niveau colonne : - `fiscal_events.event_type IN ('N','P','T','C','I','F','S','R')` - rejette toute insertion avec un label invalide. - `orders.status IN ('open','closed','voided')` - rejette tout statut inconnu. - `orders.order_type IN ('dine_in','takeaway')` - rejette tout type inconnu. - `menu_items.vat_code IN ('A','B','C','D','X')` - rejette tout code TVA invalide. - `payments.method IN ('cash','card','other')` - restreint les modes de paiement. - `users.role IN ('admin','cashier','waiter')` - restreint les rôles à ceux mappés sur les profils SCE 2.0. - `peppol_deliveries.status IN ('queued','accepted','failed')` - restreint les statuts de livraison Peppol.

1.4 Contraintes d'unicité

- `fiscal_events UNIQUE(event_type, sequence_number)` - empêche la double insertion du même couple (type, compteur). Couplée à la monotonie du compteur, garantit la stricte séquentialité exigée par la spec SCE 2.0.
- `users` primary key sur `insz` - un même INSZ ne peut pas être enregistré deux fois.
- `translations UNIQUE(entity_type, entity_id, field, lang)` - une seule traduction par couple entité / champ / langue.

2. Mécanismes au niveau applicatif

2.1 Append-only pour **fiscal_events**

Le code applicatif ne contient AUCUNE instruction `DELETE FROM fiscal_events` ni `UPDATE fiscal_events SET ...` (vérification grep - cf. `DECLARATION_FONCTIONS_INTERDITES_FR.md` §1 et §2). Les seules opérations possibles sont :

- `INSERT` via `pos_fiscal::archive::archive_event` (`crates/pos-fiscal/src/archive.rs:38`).
- `SELECT` via les fonctions de `pos_db::queries::fiscal`.

Cette discipline est garantie par revue de code : tout patch introduisant un DELETE/UPDATE sur `fiscal_events` est rejeté en pull request (politique organisationnelle, cf. `PROFIL_ENTREPRISE_FR.md`).

2.2 Transaction atomique compteur + archive

Chaque événement fiscal est inséré dans une transaction `BEGIN IMMEDIATE` qui inclut :

1. L'allocation du compteur (`FiscalCounter::next` - incrément persistant).
2. L'insertion de la ligne `fiscal_events`.
3. (Le cas échéant) la fermeture de la commande (`UPDATE orders SET status = 'closed'`).

Si l'une de ces opérations échoue, toute la transaction est rejouée à la racine : ni le compteur ni l'archive ni le statut de commande ne sont modifiés. Voir `COUNTER_STRATEGY.md` §3 et la régression `moyun-pos::spf_e2e_no_gap`.

2.3 Aucune suppression de compteur

Le code applicatif ne contient AUCUNE instruction `DELETE FROM fiscal_counters` ni `UPDATE fiscal_counters SET current_value = ...` ailleurs que dans `FiscalCounter::next` (qui incrémente exclusivement, jamais ne diminue). Voir `DECLARATION_FONCTIONS_INTERDITES_FR.md` §4.

2.4 Validation des entrées

- INSZ validé par `pos_core::user::validate_insz` (11 chiffres ASCII + mod-97) avant tout appel commande mutant un événement fiscal.
- Montants exprimés en centimes en **`i64`** - pas de flottants pour les calculs monétaires. Toute conversion depuis l'interface utilisateur passe par un parsing strict (panique si overflow ou caractères invalides).
- Horloge système non modifiable par l'opérateur (image kiosque, pas d'accès root, NTP via `chrony`).

3. Mécanismes au niveau système

3.1 Permissions de fichiers

Fichier	Propriétaire	Mode	Justification
`/var/lib/moyun-pos/pos.db`	`moyun-pos:moyun-pos`	`0640`	Lecture/écriture par le service uniquement
`/var/lib/moyun-pos/pos.db-wal`	`moyun-pos:moyun-pos`	`0640`	Idem
`/var/lib/moyun-pos/pos.db-shm`	`moyun-pos:moyun-pos`	`0640`	Idem
`/var/lib/moyun-pos/backups/`	`moyun-pos:moyun-pos`	`0750`	Répertoire des sauvegardes
`/var/lib/moyun-pos/backups/pos-*.db`	`moyun-pos:moyun-pos`	`0640`	Sauvegardes journalières

Fichier	Propriétaire	Mode	Justification
`/var/lib/moyun-pos/logs/pos.log.*`	`moyun-pos:moyun-pos`	`0640`	Journaux applicatifs
`/etc/moyun-pos/pos.json`	`root:moyun-pos`	`0640`	Configuration, lecture par le service uniquement
`/opt/moyun-pos/moyun-pos`	`root:root`	`0755`	Binaire, exécution publique

3.2 Utilisateur de service dédié

Le service `moyun-pos.service` (systemd) tourne sous l'utilisateur non-root `moyun-pos`, créé à l'installation de l'image kiosque. Il n'a PAS de shell de connexion (`/usr/sbin/nologin`) ni de mot de passe.

```
[Service]
User=moyun-pos
Group=moyun-pos
ExecStart=/opt/moyun-pos/moyun-pos
NoNewPrivileges=yes
ProtectSystem=strict
ReadWritePaths=/var/lib/moyun-pos
PrivateTmp=yes
ProtectHome=yes
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectControlGroups=yes
RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6
SystemCallFilter=@system-service
```

L'opérateur n'a aucun moyen de s'élever en `root` (cf. §3.4 ci-dessous), donc aucun moyen de manipuler les permissions ou d'écrire ailleurs que dans `/var/lib/moyun-pos/`.

3.3 Image scellée

L'image kiosque livrée est scellée :

- Pas de serveur SSH.
- Pas de console TTY active (`agetty` désactivé sur tous les VT).
- Pas d'outils administrateur : `sqlite3`, `vim`, `nano`, `less`, `cat`, `cp`, `mv` sont absents de l'image (utilisation de busybox minimal pour le système).
- Cage Wayland compositor démarre directement sur moyun-pos en plein écran, sans gestionnaire de fenêtres ni écran de connexion.

Cf. `MANUEL_INSTALLATION_FR.md` pour les détails de construction d'image.

3.4 Pas d'élévation de privilèges

- Pas de `sudo` configuré sur l'image kiosque.
- L'utilisateur `moyun-pos` n'est dans aucun groupe sensible (`wheel`, `sudo`, `adm`…).
- L'opérateur ne dispose même pas d'un compte shell ; l'écran tactile démarre moyun-pos en kiosk-mode.

3.5 Boot sécurisé

- BIOS verrouillé par mot de passe (à configurer à l'installation cf. `KIOSK_DEPLOYMENT_CHECKLIST.md`).
- Boot uniquement depuis le disque interne (USB / réseau désactivés).
- Cela empêche un opérateur de booter sur une clé USB live pour monter et modifier la base.

4. Mécanismes au niveau cryptographique

4.1 Signature ECDSA secp256r1 (par le FDM)

Chaque événement fiscal porte la signature `digitalSignature` du FDM dans `fiscal_events.fdm_response_json`. Cette signature couvre le payload de la requête. Toute modification a posteriori d'une ligne `fiscal_events` (DELETE + INSERT avec contenu différent, par exemple via accès physique) briserait la signature : une re-vérification par `pos_fiscal::signature::verify_signature` avec la clé publique du FDM échouerait, prouvant la falsification.

4.2 Hash SHA-1 du binaire

Le hash SHA-1 du binaire `moyun-pos` certifié est consigné dans le formulaire de demande de certification. Le contrôleur peut à tout moment vérifier sur le kiosque :

```
sha1sum /opt/moyun-pos/moyun-pos
```

Toute différence avec la valeur consignée prouverait que le binaire exécuté n'est pas celui certifié.

4.3 Hash du code source

Tag git signé du commit de la révision certifiée. Permet de reproduire le binaire à l'identique et de vérifier l'absence de modifications post-certification.

5. Sauvegardes

Voir `FEUILLE_ACQUISITION_DONNEES_FR.md` §1.2.

Politique : - Sauvegarde automatique avant chaque Z-report (via SQLite Online Backup API). - Fichier daté à la seconde, jamais écrasé. - Stockage local - pas de sauvegarde cloud automatisée (le distributeur peut configurer une copie distante si demandé par le commerçant, en dehors du périmètre certifié). - Rétention indéfinie (au moins 7 ans, exigence légale belge).

6. Confidentialité (GDPR)

Voir `FEUILLE_ACQUISITION_DONNEES_FR.md` §6 et `PROFIL_ENTREPRISE_FR.md` §GDPR.

- INSZ stocké en clair (donnée personnelle obligatoire fiscalement).
- PIN stocké en hash bcrypt (jamais en clair).
- Pas de données client identifiantes côté POS sauf VAT number du client B2B pour les factures Peppol.
- Pas d'export automatique de données hors POS+FDM.

7. Audit trail croisé

Chaque événement fiscal est tracé à TROIS endroits indépendants :

1. Côté POS : table `fiscal_events`.
2. Côté FDM : journal interne du FDM (signe et stocke indépendamment).
3. Côté SPF Finances : remontée cloud quotidienne du FDM via le protocole `FPSFIN_API_PROTOCOL_FDM_FINCLOU`

Le contrôleur peut croiser ces trois sources pour repérer toute divergence. Une suppression locale de `fiscal_events` serait immédiatement visible par la non-correspondance avec le journal FDM.